

Atividades capítulo 2 e 3 - Livro de algoritmos Cormen 2 edição

Antônio Erick Freitas Ferreira

Instituto de Computação - Unicamp

10 de março de 2026

Capítulo 2

Exercício 2.1-3

Considere o **problema de pesquisa**:

Entrada: Uma sequência de n números $A = \{a_1, a_2, \dots, a_n\}$ e um valor v

Saída: Um índice tal que $v = A[i]$ ou o valor especial NIL, se v não aparecer em A .

Escreva um pseudocódigo para **pesquisa linear**, que faça a varredura da sequência, procurando por v . Usando um loop invariante, prove que seu algoritmo é correto. Certifique-se de que seu loop invariante satisfaz às três propriedades necessárias.

Resposta:

Algorithm 1: Pesquisa Linear

Input: Vetor A de tamanho n e um número v

Output: Índice ou NIL

```
1 for  $i \leftarrow 1$  to  $\text{comprimento}[A]$  do
2   if  $A[i] = v$  then
3     return  $i$ 
4 return NIL
```

Invariante de Laço: No início de cada iteração do laço for das linhas 1-3, o valor v não está presente no subarranjo $A[0, \dots, i - 1]$.

Inicialização: No início, com $i = 1$, o subarranjo $A[1, \dots, i - 1]$ é vazio, então é verdade que v não aparece em nenhuma posição de $A[i, \dots, i - 1]$.

Manutenção: Como v não aparece em $A[1, \dots, i - 1]$, na iteração atual o algoritmo compara $A[i]$ com v :

- Se $A[i] = v$, então o algoritmo retorna i e termina, garantindo a correção: o índice retornado de fato contém v .
- Caso contrário, $A[i]$ é diferente de v e i é incrementado para $i + 1$.

Assim, após o incremento, o novo intervalo passa a ser $A[1, \dots, i]$, entretanto, como v não está em $A[i, \dots, i - 1]$, como mostrado na inicialização, e $A[i] \neq v$, mantém-se que v não aparece em $A[1, \dots, i]$. Portanto, o loop invariante é mantido válido para o novo subarranjo.

Término: O laço acaba quando $i = n + 1$, onde $n = \text{comprimento}[A]$ ou v é encontrado durante a execução. Porém, pelo loop invariante, temos que v não aparece em $A[1, \dots, n]$. Como o algoritmo teria retornado o índice caso v estivesse no arranjo, conclui-se que $v \notin A$ e o retorno NIL está correto.

Exercício 2.1-4

Considere o problema de somar dois inteiros binários de n bits, armazenados em dois arranjos de n elementos A e B . A soma dos dois inteiros deve ser armazenada em forma binária em um arranjo de $(n + 1)$ elementos C . Enuncie o problema de modo formal e escreva o pseudocódigo para somar os dois inteiros.

Resposta:

Dado dois números binários de n bits representados pelos arranjos A e B , apresente o pseudocódigo que realiza a soma desses dois números, armazenando o resultado em um novo arranjo C também binário de tamanho $n + 1$.

Algorithm 2: Soma Binária

Input: Vetores A e B
Output: Vetor C

```

1  $i \leftarrow \text{comprimento}[A]$ 
2  $carry \leftarrow 0$ 
3  $C[0, 0, \dots, i + 1] \leftarrow 0$ 
4 while  $i > 0$  do
5    $k \leftarrow i + 1$ 
6    $C[k] \leftarrow A[i] + B[i] + carry$ 
7    $carry \leftarrow C[k] \div 2$ 
8    $C[k] \leftarrow C[k] \bmod 2$ 
9    $i \leftarrow i - 1$ 
10  $C[0] \leftarrow carry$ 
11 return  $C$ 
```

Exercício 2.2-2

Considere a ordenação de n números armazenados no arranjo A , localizando primeiro o menor elemento de A e permutando esse elemento com o elemento contido em $A[1]$. Em seguida, encontre o segundo menor elemento de A e o troque pelo elemento $A[2]$. Continue dessa maneira para os primeiros $n - 1$ elementos de A . Escreva o pseudocódigo para esse algoritmo, conhecido como **ordenação por seleção**. Que loop invariante esse algoritmo mantém? Por que ele só precisa ser executado para os primeiros $n - 1$ elementos, e não para todos os n elementos? Forneça os tempos de execução do melhor caso e do pior caso da ordenação por seleção em notação Θ .

Resposta:

Algorithm 3: Ordenação por seleção

Input: Vetor A
Output: Vetor A ordenado de forma crescente

```

1  $i \leftarrow 1$ 
2 while  $i \leq \text{comprimento}[A] - 1$  do
3    $min \leftarrow i$ 
4    $k \leftarrow i + 1$ 
5   while  $k \leq \text{comprimento}[A]$  do
6     if  $A[k] < A[min]$  then
7        $min \leftarrow k$ 
8    $k \leftarrow k + 1$ 
9    $temp \leftarrow A[min]$ 
10   $A[min] \leftarrow A[i]$ 
11   $A[i] \leftarrow temp$ 
12   $i \leftarrow i + 1$ 
13 return  $A$ 
```

Invariante de laço: No início da i -ésima iteração, o subarranjo $A[1, \dots, i - 1]$ está ordenado em ordem crescente e contém os $i - 1$ menores elementos do arranjo.

Exatamente por conta da invariante de laço não é necessário executar o algoritmo para os n elementos, pois quando $i = n$ a invariante de laço garante que o subarranjo $A[1, \dots, i - 1]$ está ordenado com este subarranjo contendo os $i - 1$ menores elementos. Desta forma, obrigatoriamente o n -ésimo elemento é o maior elemento do arranjo completo e já está na posição correta.

Descobrimos $T(n)$:

Linha 1 tem tempo constante c_1

Linha 2 tem tempo $c_2 \cdot n$

Linha 3 tem tempo $c_3 \cdot (n - 1)$

Linha 4 tem tempo $c_4 \cdot (n - 1)$

Linha 5 tem tempo $c_5 \cdot \sum_{i=1}^{n-1} (n - i + 1)$

Linha 6 tem tempo $c_6 \cdot \sum_{i=1}^{n-1} (n - i)$

Linha 7 tem tempo $c_7 \cdot \sum_{i=1}^{n-1} (n - i)$

Linha 8 tem tempo $c_8 \cdot \sum_{i=1}^{n-1} (n - i)$

Linha 9 tem tempo $c_9 \cdot (n - 1)$

Linha 10 tem tempo $c_{10} \cdot (n - 1)$

Linha 11 tem tempo $c_{11} \cdot (n - 1)$

Linha 12 tem tempo $c_{12} \cdot (n - 1)$

Linha 13 tem tempo constante c_{13}

Desta forma temos que

$$T(n) = c_1 + c_2 \cdot n + c_3 \cdot (n - 1) + c_4 \cdot (n - 1) + c_5 \cdot \sum_{i=1}^{n-1} (n - i + 1) + c_6 \cdot \sum_{i=1}^{n-1} (n - i) + c_7 \cdot \sum_{i=1}^{n-1} (n - i) + c_8 \cdot \sum_{i=1}^{n-1} (n - i) + c_9 \cdot (n - 1) + c_{10} \cdot (n - 1) + c_{11} \cdot (n - 1) + c_{12} \cdot (n - 1) + c_{13}$$

Sabemos que

$$\sum_{i=1}^{n-1} (n - i + 1) = \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i + \sum_{i=1}^{n-1} 1 = n(n - 1) - \frac{(n-1)((n-1)+1)}{2} + (n - 1) = \frac{n^2 + n - 2}{2}$$

$$\sum_{i=1}^{n-1} n - i = \sum_{i=1}^{n-1} n - \sum_{i=1}^{n-1} i = n(n - 1) - \frac{(n-1)((n-1)+1)}{2} = \frac{n^2 - n}{2}$$

Se o arranjo estiver ordenado em ordem inversa, isto é, em ordem decrescente, resulta o pior caso. Neste caso, o algoritmo apenas acabará quando realizar todas as suas extensas operações, portanto:

$$T(n) = c_1 + c_2 n + c_3 n - c_3 + c_4 n - c_4 + c_5 \frac{n^2 + n - 2}{2} + c_6 \frac{n^2 - n}{2} + c_7 \frac{n^2 - n}{2} + c_8 \frac{n^2 - n}{2} + c_9 n - c_9 + c_{10} n - c_{10} + c_{11} n - c_{11} + c_{12} n - c_{12} + c_{13}$$

Como o termo de maior grau é n^2 , então $T(n) = \Theta(n^2)$.

Se o arranjo estiver ordenado em ordem crescente teremos o melhor caso. Neste caso, $T(n)$ segue com os mesmos valores, exceto pela linha 7 que nunca irá ser executada, portanto:

$$T(n) = c_1 + c_2 n + c_3 n - c_3 + c_4 n - c_4 + c_5 \frac{n^2 + n - 2}{2} + c_6 \frac{n^2 - n}{2} + c_8 \frac{n^2 - n}{2} + c_9 n - c_9 + c_{10} n - c_{10} + c_{11} n - c_{11} + c_{12} n - c_{12} + c_{13}$$

Como o termo de maior grau segue sendo n^2 , então $T(n) = \Theta(n^2)$.

Exercício 2.2-3

Considere mais uma vez a **pesquisa linear** (Algoritmo 1). Quantos elementos da sequência de entrada precisam ser verificados em média, supondo-se que o elemento que está sendo procurado tenha a mesma probabilidade de ser qualquer elemento do arranjo? E no pior caso? Quais são os tempos de execução do caso médio e do pior caso da pesquisa linear em notação Θ ? Justifique sua resposta.

Resposta:

Descobrimos $T(n)$:

Linha 1 tem tempo $c_1 \cdot (n + 1)$

Linha 2 tem tempo $c_2 \cdot n$

Linha 3 tem tempo c_3

Linha 4 tem tempo c_4

As linhas 3 e 4 dependerá dos seguintes cenários: o elemento buscado é o último da lista ou o elemento buscado não está no arranjo. É importante entender que quando um caso ocorre o outro obrigatoriamente não ocorre. Entretanto, provarei a seguir que em ambos os casos a complexidade se mantém a mesma.

Para o pior caso onde o elemento é o último da lista:

$$T(n) = c_1(n + 1) + c_2 n + c_3 = c_1 n + c_1 + c_2 n + c_3 = (c_1 + c_2)n + (c_1 + c_3), \text{ que pode ser visto como } an + b.$$

Para o pior caso onde o elemento não está na lista:

$$T(n) = c_1(n + 1) + c_2 n + c_4 = c_1 n + c_1 + c_2 n + c_4 = (c_1 + c_2)n + (c_1 + c_4), \text{ que pode ser visto como } an + b.$$

Como ambos os casos $T(n)$ pode ser expressado como uma função linear de n , então $T(n) = \Theta(n)$ para o pior caso.

Para o caso médio, temos que a linha 3 irá acontecer, portanto:

$$T(n) = c_1 n + c_2 n + (c_1 + c_3)$$

Entretanto, como cada elemento tem chance $\frac{1}{n}$ de estar na i -ésima posição, então a linha 2 será executada i vezes, sendo uma comparação por elemento até encontrá-lo. Desta forma, temos:

$$\sum_{i=1}^n (i \cdot \frac{1}{n}) = \frac{1}{n} \cdot \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

Substituindo na função do tempo, temos:

$$T(n) = c_1 \frac{n+1}{2} + c_2 \frac{n+1}{2} + (c_1 + c_3) = (c_1 + c_2) \frac{n+1}{2} + (c_1 + c_3), \text{ que pode ser visto como } an + b.$$

Portanto, $T(n)$ pode ser expressado como uma função linear de n , então $T(n) = \Theta(n)$ para o caso médio.

Exercício 2.3-3

Use indução matemática para mostrar que, quando n é uma potência exata de 2, a solução da recorrência

$$T(n) = \begin{cases} 2 & \text{se } n = 2, \\ 2T(n/2) + n & \text{se } n = 2^k, \text{ para } k \geq 1 \end{cases}$$

é $T(n) = n \log n$

Resposta:

Caso base: Quando $k = 1$, temos $n = 2^1$, dado que n é uma potência de 2. Então $n = 2$.

$$T(2) = 2 \cdot \log_2 2 = 2.$$

Hipótese de indução: Quando $n = 2^k$, para $k \geq 1$, $T(2^k) = 2^k \log_2 2^k = 2^k \cdot k$

Queremos provar que quando $n = 2^{k+1}$, temos $T(n) = n \log n$

$$T(2^{k+1}) = 2T\left(\frac{2^{k+1}}{2}\right) + 2^{k+1} \tag{1}$$

$$= 2T\left(\frac{2^k \cdot 2}{2}\right) + 2^{k+1} \tag{2}$$

$$= 2T(2^k) + 2^{k+1} \quad (\text{Aplicando a hipótese de indução em } T(2^k)) \tag{3}$$

$$= 2 \cdot 2^k \cdot k + 2^{k+1} \tag{4}$$

$$= 2^{k+1} k + 2^{k+1} \tag{5}$$

$$= 2^{k+1} \cdot (k + 1) \quad (\text{Observe que } \log_2 2^{k+1} = (k + 1)) \tag{6}$$

$$= 2^{k+1} \cdot \log_2 2^{k+1} \quad (\text{Como } n = 2^{k+1}) \tag{7}$$

$$= n \log_2 n \tag{8}$$

Exercício 2.3-5

Voltando para o problema da pesquisa (ver Algoritmo 1) observe que, se a sequência A estiver ordenada, poderemos comparar o ponto médio da sequência com v e eliminar metade da sequência de consideração posterior. A **pesquisa binária** é um algoritmo que repete esse procedimento, dividindo ao meio o tamanho da porção restante da sequência a cada vez. Escreva pseudocódigo, sendo ele iterativo ou recursivo, para pesquisa binária. Demostre que o tempo de execução do pior caso da pesquisa binária em $\Theta(\log n)$.

Resposta:

Algorithm 4: Pesquisa_binaria

Input: Vetor A , dois números p e r , que representam o primeiro e o último de A respectivamente, e um número v
Output: Índice de v se $v \in A$ ou NIL se $v \notin A$

```
1 if  $p > r$  then
2   return NIL
3 else
4    $meio \leftarrow p + \lfloor \frac{(r-p)}{2} \rfloor$ 
5   if  $A[meio] = v$  then
6     return  $meio$ 
7   else if  $A[meio] > v$  then
8     return Pesquisa_binaria( $A, p, meio - 1, v$ )
9   else
10    return Pesquisa_binaria( $A, meio + 1, r, v$ )
```

Se $v \notin A$ ou v for o último elemento buscado, resultará no pior caso. Nestes casos, o algoritmo irá parar somente quando atingir o caso onde a linha 2 ou 6 é executada. Os casos citados se diferenciam apenas no fato de que na última iteração o algoritmo decidirá se v está ou não em A .

Descobrimos $T(n)$:

Sabendo que as operações que ocorrem antes da subchamada recursiva possuem custo constante $1 = \Theta(1)$ e que o algoritmo reduz o espaço de busca pela metade a cada chamada recursiva, temos a seguinte relação de recorrência:

$$T(n) = \begin{cases} 1 & \text{se } n = 1, \\ T(n/2) + 1 & \text{se } n > 1 \end{cases}$$

Assumindo que n é uma potência de 2, isto é, $n = 2^k$ para $k \geq 0$, no pior caso, o algoritmo continuará dividindo o subarranjo até que o índice p seja maior que r atingindo o caso base da linha 1, o que leva a uma profundidade de recursão de aproximadamente $\log_2 n$ níveis, como mostra a equação a seguir (i representa o i -ésimo nível da árvore):

$$\frac{n}{2^i} = 1 \rightarrow n = 2^i \rightarrow \log_2 n = \log_2 2^i \rightarrow i = \log_2 n$$

Agora, basta somarmos quanto cada nível custa para chegarmos a formula final

$$\sum_{i=0}^{\log_2 n} 1 = \log_2 n$$

Portanto, podemos concluir que o Algoritmo 4, no seu pior caso, tem complexidade na ordem de $\Theta(\log_2 n)$

Exercício 2.3-6

Observe que o loop while das linhas 5 a 7 do procedimento **INSERTION-SORT** na Seção 2.1 utiliza uma pesquisa linear para varrer (no sentido inverso) o subarranjo ordenado $A[1, \dots, j-1]$. Podemos usar em vez disso uma pesquisa binária para melhorar o tempo de execução global do pior caso da ordenação por inserção para $\Theta(n \log n)$?

Algorithm 5: INSERTION-SORT

Input: Vetor A
Output:

```
1 for  $j \leftarrow 2$  to comprimento[ $A$ ] do
2    $chave \leftarrow A[j]$ 
3    $i \leftarrow j - 1$ 
4   while  $i > 0$  e  $A[i] > chave$  do
5      $A[i+1] \leftarrow A[i]$ 
6      $i \leftarrow i - 1$ 
7    $A[i+1] \leftarrow chave$ 
```

Observando o algoritmo acima, podemos perceber que o while da linha 4 realiza 2 operações de forma simultânea: busca o índice correto da inserção do j -ésimo elemento e faz shift em todos os elementos, isto

é, o i -ésimo elemento irá para a posição $i+1$.

A pesquisa binária poderia ajudar a encontrar o índice correto em tempo $\Theta(\log_2 n)$. Entretanto, ainda seria necessário realizar a operação de shift nos elementos entre j e i . Veja o algoritmo abaixo (nele supomos que há uma função chamada *Binary_Search* que recebe um arranjo, o primeiro e o último índice, e um valor. Por fim, esta função retorna o índice onde o valor deve ser inserido com complexidade, no pior caso, de $\Theta(\log_2 n)$).

Algorithm 6: INSERTION-SORT-With-Binary-Search

Input: Vetor A

Output:

```
1 for  $j \leftarrow 2$  to  $\text{comprimento}[A]$  do
2    $\text{chave} \leftarrow A[j]$ 
3    $i \leftarrow j - 1$ 
4    $\text{index} \leftarrow \text{Binary\_Search}(A, 0, i, \text{chave})$ 
5   while  $i \geq \text{index}$  do
6      $A[i + 1] \leftarrow A[i]$ 
7      $i \leftarrow i + 1$ 
8    $A[\text{index}] \leftarrow \text{chave}$ 
```

Embora a busca binária possa encontrar mais rapidamente o índice onde o j -ésimo elemento deve ser inserido, a operação de shift continua dominando assintoticamente a função. Portanto, não é possível melhorar o INSERTION-SORT utilizando da busca binária para conseguir a complexidade $\Theta(n \log n)$.

Exercício 2.3-7

Descreva um algoritmo de tempo $\Theta(n \log n)$ que, dado um conjunto S de n inteiros e outro inteiro x , determina se existem ou não dois elementos em S cuja a soma seja exatamente x .

Resposta:

Para este algoritmo, usarei do MERGE-SORT que está escrito e analisado na Seção 2.3.1.

Algorithm 7: Busca-Soma

Input: Conjunto S , inteiro x

Output: true, se houver 2 elementos em S cujo a soma é x , caso contrário, false

```
1  $i \leftarrow$  primeiro índice de  $S$ 
2  $k \leftarrow$  último índice de  $S$ 
3 MERGE-SORT( $S, i, k$ )
4 while  $i < k$  do
5   if  $S[i] + S[k] = x$  then
6     return true
7   else if  $S[i] + S[k] < x$  then
8      $i \leftarrow i + 1$ 
9   else
10     $k \leftarrow k - 1$ 
11 return false
```

Como a operação de ordenação tem complexidade, no pior caso, de $\Theta(n \log n)$ e os passos em busca de x tem complexidade, no pior caso, de $\Theta(n)$, então a operação de ordenação irá dominar assintoticamente a complexidade desta função (em seu âmbito completo). Portanto, esta função tem complexidade, no pior caso, $\Theta(n \log n)$.

Problema 2-1 Ordenação por inserção sobre arranjos pequenos na ordenação por intercalação

Embora a ordenação por intercalação funcione no tempo de pior caso $\Theta(n \log n)$ e a ordenação por inserção funcione no tempo de pior caso $\Theta(n^2)$, os fatores constantes na ordenação por inserção a tornam mais rápida para n pequeno. Assim, faz sentido usar a ordenação por inserção dentro da ordenação por intercalação quando os subproblemas se tornam suficientemente pequenos. Considere uma modificação na ordenação por intercalação, na qual n/k sublistas de comprimento k são ordenadas usando-se a ordenação por inserção, e depois intercaladas com o uso do mecanismo padrão de intercalação, onde k é um valor a

ser determinado.

a) Mostre que as n/k sublistas, cada uma com comprimento k , podem ser ordenadas através da ordenação por inserção no tempo de pior caso $\Theta(nk)$

Resposta:

Como teremos n/k sublistas, todas de tamanho k , sendo ordenadas, no pior caso, em tempo $\Theta(k^2)$, podemos ver a seguinte equação:

$$\frac{n}{k} \cdot \Theta(k^2) = \frac{n}{k} \cdot k^2 = n \cdot k = \Theta(nk)$$

b) Mostre que as sublistas podem ser intercaladas no tempo de pior caso $\Theta(n \log(\frac{n}{k}))$

Resposta:

O algoritmo de intercalação constrói uma árvore de chamadas recursivas até o problema chegar ao caso base, por isso tem sua complexidade $\Theta(\log n)$ e usa n operações para realizar a ordenação, isso justifica sua complexidade de pior caso ser $\Theta(n \log n)$. Entretanto, nesta variável sugerida no enunciado, teremos que cada sublista será dividida até seu tamanho ser igual a k . Com isso, teremos $\log(\frac{n}{k})$ níveis que ainda necessitarão de n operações de ordenação, então sua complexidade, em pior caso, será de $\Theta(n \log(\frac{n}{k}))$.

c) Dado que o algoritmo modificado é executado no tempo de pior caso $\Theta(nk + n \log(\frac{n}{k}))$, qual é o maior valor assintótico (notação Θ) de k como uma função de n para a qual o algoritmo modificado tem o mesmo tempo de execução assintótico que a ordenação por intercalação padrão?

Resposta:

Queremos encontrar um k em função de n de forma que ao aplicarmos k em $nk + \log(\frac{n}{k}) = n \log(n)$.

$$nk + \log(\frac{n}{k}) = n \log(n) \text{ implica que } nk \leq n \log(n) \rightarrow k \leq \log(n)$$

Usando $k = \log n$, teremos:

$$n \log n + \log(\frac{n}{\log n}) = n \log n + n(\log n - \log \log n) = n \log n + n \log n - n \log \log n = 2n \log n - n \log \log n$$

Como sabemos que $n \log n > n \log \log n$, então, com $k = \log n$ teremos $\Theta(n \log n)$. Portanto, o maior valor para k é $\log n$.

d) Como k deve ser escolhido na prática?

Na prática k deve ser uma potência de 2, como 2^i , com $0 \leq i \leq 5$. Isso ajuda a manter o alinhamento com a estrutura do MERGE-SORT e também mantém baixo custo no INSERTION-SORT